



INTEGRANTES:

- Melissa Guerra
- Jorge Yanez
- Stiven Viscaino

TEMA: Optimizador de horarios con CSP y búsqueda local.

OBJETIVO: Desarrollar un sistema funcional que genere y optimice horarios académicos aplicando CSP, backtracking y búsqueda local, separando restricciones duras y blandas.

1. PRESENTACIÓN DEL PROBLEMA

Planificar un horario académico resulta ser un problema complejo debido a las restricciones o complicaciones como la disponibilidad de los docentes, uso de los espacios disponibles, y choques entre materias. Hacer este trabajo de forma manual resulta en obtener un horario poco adecuado.

Con el objetivo de evitar estos problemas al generar los horarios, se propone una aplicación de generación de horarios que usa técnicas y algoritmos de Inteligencia Artificial, en este caso específico Problemas de Satisfacción de Restricciones (CSP) y algoritmos de búsqueda local. Permitiendo generar horarios válidos, con capacidad de optimizarlos para mejorar la calidad del horario.

La aplicación:

- Evita los choques de horarios entre materias.
- Toma en cuenta la disponibilidad de horarios de los docentes, y aulas disponibles.
- Genera horarios cómodos tanto para estudiantes como para docentes.

2. REPRESENTACIÓN DEL PROBLEMA

Tabla de representación

Elemento	Contenido
Variables	Representan las sesiones semanales por materia
Dominios	Cada variable puede tomar una combinación posible: (día, hora, aula). Ejemplo: (lunes, 7AM, Aula1).
Estado inicial	Horario vacío, las sesiones no tienen día, hora, aula y docente.
Estado Final u Objetivo	Todas las sesiones están asignadas cumpliendo las restricciones duras y buscando mejorar las restricciones blandas.

Restricciones duras	1. No puede haber dos clases en la misma aula, el mismo día y a la misma hora. 2. Un profesor no puede dictar dos clases el mismo día y a la misma hora. 3. El profesor asignado debe estar capacitado para dictar esa materia. 4. El profesor debe estar disponible en el día y hora asignados. 5. Ninguna clase puede asignarse en la hora de almuerzo, por ejemplo, de 1pm a 2pm. 6. Cada materia debe tener entre 1 y 3 sesiones semanales. 7. Una misma materia no debe repetirse más de una vez en el mismo día. 8. Todas las sesiones de una misma materia deben tener el mismo profesor. 9. El aula asignada debe estar disponible en ese horario. 10. No se puede tener dos materias al mismo tiempo aun si son en aulas diferentes.
Restricciones blandas	1. Evitar clases muy temprano. 2. Evitar clases viernes en la tarde. 3. Preferir ciertas aulas para determinadas materias. 4. Reducir huecos grandes entre clases del mismo grupo o curso. 5. Distribuir las clases de forma equilibrada durante la semana.

3. DESCRIPCIÓN DEL MODELO

3.1 IMPLEMENTACIÓN DEL CÓDIGO

ALGORITMOS IMPLEMENTADOS

Backtracking para generar solución inicial

El backtracking se usa para construir el horario paso a paso. Toma una sesión sin asignar, prueba una combinación posible de día, hora y aula, y revisa si cumple las restricciones duras. Si la asignación es válida, continúa con la siguiente sesión. Si en algún punto no puede continuar, retrocede y prueba otra combinación.

Hill Climbing

Se aplica después de tener un horario válido. La idea es generar cambios pequeños en el horario, como mover una clase a otro espacio libre o intercambiar dos clases. Si el cambio mejora el puntaje, se acepta. El proceso se repite hasta que ya no se encuentra una mejora directa.

Simulated Annealing

Busca mejorar el horario, pero a diferencia de Hill Climbing puede aceptar algunos movimientos peores al inicio. Esto ayuda a escapar de óptimos locales. Conforme avanza la ejecución, la probabilidad de aceptar soluciones peores disminuye, simulando un enfriamiento progresivo.

PSEUDOCÓDIGO

INICIO

Definir horarios, materias, profesores y aulas

Generar variables:

 Cada variable representa una sesión de una materia

Generar dominios:

 Para cada variable crear combinaciones válidas:

 (día, hora, aula)

Aplicar Backtracking:

 Seleccionar variable no asignada

 Probar valores del dominio

 Verificar restricciones duras:

- No choque de aulas
- No choque de profesores
- No choque de materias
- Disponibilidad del profesor
- El estudiante no puede tener dos clases al mismo tiempo

 Si la asignación es válida:

 Continuar recursivamente

 Si no existe solución:

 Retroceder (backtrack)

Obtener solución inicial

Calcular calidad de la solución:

 Penalizar:

- Clases muy temprano
- viernes en la tarde
- Huecos en horarios

 Bonificar:

- Uso de aulas preferidas
- Distribución equilibrada

Optimizar solución:

 Opción 1: Hill Climbing

 Generar vecinos

 Elegir el vecino con mejor puntaje

 Repetir hasta llegar a un óptimo local

 Opción 2: Simulated Annealing

 Generar vecino aleatorio

 Aceptar mejores soluciones

 A veces aceptar peores soluciones

 Reducir temperatura progresivamente

Mostrar horario final en la interfaz gráfica

FIN

FUNCIONES DE LA INTERFAZ

Boton	Descripcion
Generar	Crea una solucion inicial usando backtracking.
Hill	Aplica Hill Climbing sobre el horario actual.
Annealing	Aplica Simulated Annealing sobre el horario actual.
+ Materia	Permite agregar una materia nueva con sesiones, estudiantes, aula preferida y profesor existente o nuevo.
+ Restriccion	Permite agregar una restriccion nueva, por ejemplo profesor no disponible o aula ocupada.
Reiniciar	Limpia la vista para hacer otra prueba. No borra las materias o restricciones ya agregadas.

Imagen 1. Funciones de la interfaz.

PRUEBAS Y RESULTADOS

Las pruebas se realizaron con un conjunto de datos ya ingresados:

Materia	Sesiones por semana	Profesor asignado
Programacion	3	Juan
Fisica	2	Carlos
Matematicas	2	Ana
Redes	2	Juan
Base de Datos	3	Luis
Materia agregada en demo	1 a 3	Profesor existente o nuevo

Aula	Capacidad	Uso dentro del modelo
Aula1	20	Aula normal
Aula2	30	Aula normal
Lab1	25	Laboratorio, preferido para materias practicas
Lab2	20	Laboratorio

Profesor	Materia(s) que puede dictar	Restricciones iniciales
Juan	Programacion, Redes	No disponible viernes 9AM
Ana	Matematicas	No disponible martes 11AM
Carlos	Fisica	No disponible lunes 7AM y miercoles 5PM
Luis	Base de Datos	No disponible jueves 2PM
Profesor nuevo	Materia agregada desde la interfaz	Puede tener una restriccion opcional de disponibilidad

Imagen 2. Conjunto de datos de prueba.

Caso de prueba 1: horario inicial

En este caso se genera el horario base con las materias iniciales. El sistema asigna las 12 sesiones de clase y valida que no existan restricciones duras violadas. El puntaje inicial normalmente es menor porque aún no se optimiza el horario.

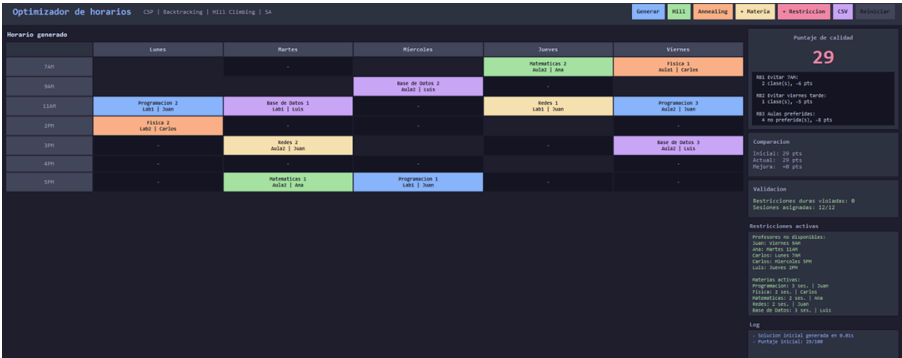


Imagen 3. Horario Inicial

Caso de prueba 2: mejora con Hill Climbing

Luego de generar la solución inicial, se aplica Hill Climbing. En una de las pruebas realizadas, el puntaje subió de 29 a 87 puntos, manteniendo 0 restricciones duras violadas. Esto muestra que la búsqueda local mejora la calidad del horario sin romper las reglas obligatorias.

Imagen 4. Hill Climbing

Caso de prueba 3: mejora con Simulated Annealing

También se probó Simulated Annealing. En una ejecución el puntaje subió aproximadamente de 29 a 84 puntos. Este resultado permite comparar los dos métodos de mejora y explicar que ambos buscan aumentar la calidad de la solución.

Imagen 5. Simulated Annealing

Caso de prueba 4: agregar una restricción nueva

Para cumplir con la demo, el sistema permite agregar una restricción nueva. Por ejemplo, se puede indicar que Ana no está disponible el miércoles a las 7AM. Luego el sistema recalcula el horario y vuelve a optimizarlo. En una prueba, después de agregar la restricción, la solución inicial fue de 21 puntos y luego con Hill Climbing subió a 83 puntos, con 0 restricciones duras violadas.

Imagen 6. Nueva Restricción.

Prueba	Solucion inicial	Solucion mejorada	Restricciones duras	Observacion
Horario base + Hill Climbing	29 pts	87 pts	0	Mejora clara del horario inicial.
Horario base + Simulated Annealing	29 pts	84 pts	0	Tambien mejora el horario, con mas iteraciones.
Restriccion nueva + Hill Climbing	21 pts	83 pts	0	Se recalcula respetando la nueva disponibilidad.
Materia nueva Ingles	21 pts aprox.	79 pts	0	El sistema asigna 14/14 sesiones incluyendo la nueva materia.

Imagen 7. Tabla comparativa de métodos.

FORTALEZAS Y LÍMITES

Una fortaleza del sistema es que permite ver de manera directa si el horario cumple las restricciones duras. La interfaz también ayuda a defender el proyecto porque muestra el horario, el puntaje y la comparación entre situación inicial y mejorada. Además, se puede agregar una restricción o una materia nueva, lo que demuestra que no funciona solo con un ejemplo fijo.

Como límite, el sistema está pensado para un caso académico pequeño o mediano. Si se agregaran muchas materias, varios grupos y muchos docentes, sería necesario mejorar la estrategia de búsqueda y tal vez cargar los datos desde archivos externos. Aun así, para el alcance del proyecto, el programa cumple con el objetivo y permite demostrar los conceptos de CSP y búsqueda local.

CONCLUSIONES Y DISCUSIÓN

- El sistema permite generar horarios académicos válidos usando un modelo CSP, donde cada sesión de clase se asigna respetando restricciones duras.
- Backtracking fue útil para encontrar una solución inicial sin choques de profesor, aula, grupo ni disponibilidad.
- Hill Climbing y Simulated Annealing permitieron mejorar el puntaje de calidad del horario, demostrando la utilidad de la búsqueda local.
- La separación entre restricciones duras y blandas ayuda a entender mejor el problema: unas reglas son obligatorias y otras sirven para mejorar la comodidad del horario.
- La interfaz gráfica facilita la demo, ya que permite generar, optimizar, agregar restricciones y agregar materias sin cambiar el código.
- El proyecto demuestra que un problema real de asignación de horarios puede resolverse con técnicas de IA vistas en clase, no solo con una tabla hecha manualmente.

Como mejoras posibles se podría permitir que el usuario marque restricciones como obligatorias o preferidas desde la interfaz, cargar los datos desde un archivo externo para manejar casos más grandes.